

Guião 13 – WBEM com SFCB e Revisão do Semestre

Infraestruturas de Computação na Nuvem, Aula 13

1 Objetivos

- Correr um CIMOM local (SFCB) num container Docker e consultá-lo com um cliente WBEM.
- Explorar classes CIM e a relação entre elas, contrastando com o modelo plano do SNMP.
- Consolidar, através de um roteiro de autoavaliação, os conceitos práticos de todo o semestre em preparação para o teste escrito.

2 Pré-requisitos e Material

- Uma VM Linux no Proxmox (pode ser a `icn-lab` do Guião 01), com acesso `sudo`.
- Python 3 e `pip` instalados na VM.
- Acesso às pastas de todos os guiões anteriores (Aulas 01 a 12).
- Opcionalmente, credenciais de leitura da API do Proxmox, para a Parte 2.

3 Parte 1 – Explorar um CIMOM com WBEM

3.1 Introdução

Vamos instalar o SFCB (Small Footprint CIM Broker), uma implementação leve de CIMOM, dentro da nossa VM, e consultá-lo a partir de um cliente Python baseado em `pywbem`. O objetivo é observar na prática a diferença entre o modelo orientado a objetos do CIM e o modelo plano de OIDs do SNMP visto na Aula 12 – em particular, as *associações*, que o SNMP não representa.

3.2 Passo a Passo

1. **Instalar o CIMOM e os providers** na VM:

```
$ sudo apt update
$ sudo apt install -y sfcbl sblim-cmpi-base cim-schema wbemcli
$ sudo systemctl enable --now sfcbl
$ sudo systemctl status sfcbl
```

O pacote `sblim-cmpi-base` fornece os *providers* que traduzem entre o modelo CIM e o sistema real – sem eles, o CIMOM não tem dados para expor.

2. **Testar com o cliente de linha de comandos `wbemcli`**, antes de escrever código:

```
$ wbemcli -nl ein 'https://root:<password>@localhost:5989/root/cimv2:
  CIM_OperatingSystem'
```

A opção `-nl` desativa a verificação do certificado (autoassinado). Se a autenticação falhar, confirme a configuração de utilizadores em `/etc/sfcb/sfcb.cfg`; consoante a instalação, pode ser necessário usar as credenciais locais do sistema.

Nota: o SFCB e os seus providers têm empacotamento variável entre versões de distribuição. Se algum passo falhar, registre a mensagem de erro obtida – a docente indicará a alternativa (outra imagem, ou dados de demonstração), e a análise do erro conta como parte do trabalho.

3. Instalar o cliente Python:

```
$ sudo apt install -y python3-venv
$ python3 -m venv ~/wbem-env
$ ~/wbem-env/bin/pip install pywbem
```

4. Criar o script `explorar_cim.py`, que liga ao CIMOM e enumera classes:

```
import pywbem

conn = pywbem.WBEMConnection(
    'https://localhost:5989',
    ('root', 'PASSWORD'),
    default_namespace='root/cimv2',
    no_verification=True
)

print("=== Classes CIM disponiveis (root/cimv2) ===")
for cls in sorted(conn.EnumerateClassNames()):
    print(cls)
```

```
$ ~/wbem-env/bin/python explorar_cim.py
```

5. Listar instâncias e propriedades de uma classe de sistema:

```
import pywbem

conn = pywbem.WBEMConnection(
    'https://localhost:5989',
    ('root', 'PASSWORD'),
    default_namespace='root/cimv2',
    no_verification=True
)

for inst in conn.EnumerateInstances('CIM_OperatingSystem'):
    print(f"--- Instancia: {inst.classname} ---")
    for prop, value in inst.items():
        print(f" {prop}: {value}")
```

Compare os nomes das propriedades com os OIDs numéricos usados na Aula 12: aqui são autodescritivos e tipados.

6. Navegar por associações – o exercício central. Esta é a operação que não tem equivalente no SNMP. A partir de uma instância, descubra os objetos que lhe estão associados:

```
import pywbem

conn = pywbem.WBEMConnection(
    'https://localhost:5989',
```

```

    ('root', 'PASSWORD'),
    default_namespace='root/cimv2',
    no_verification=True
)

# Obter a instancia do sistema
sistemas = conn.EnumerateInstanceNames('CIM_ComputerSystem')
if not sistemas:
    raise SystemExit("Nenhuma instancia de CIM_ComputerSystem encontrada")

sistema = sistemas[0]
print(f"Sistema: {sistema}\n")

# Que associacoes partem deste objeto?
print("=== Associacoes (References) ===")
for ref in conn.ReferenceNames(sistema):
    print(f" {ref.classname}")

# Que objetos estao associados a este?
print("\n=== Objetos associados (Associators) ===")
for assoc in conn.AssociatorNames(sistema):
    print(f" {assoc.classname}")

```

Registre as classes de associação encontradas. Para cada uma, procure perceber que relação do mundo real representa.

7. Invocar informação de uma classe para ver os seus métodos, propriedades e chaves:

```

import pywbem

conn = pywbem.WBEMConnection(
    'https://localhost:5989', ('root', 'PASSWORD'),
    default_namespace='root/cimv2', no_verification=True)

cls = conn.GetClass('CIM_OperatingSystem', IncludeQualifiers=True)
print("Propriedades-chave:")
for nome, prop in cls.properties.items():
    if 'Key' in prop.qualifiers:
        print(f" {nome}")
print("\nMetodos definidos:")
for nome in cls.methods:
    print(f" {nome}")

```

Repare que a classe define *métodos*, e não apenas valores – outra diferença estrutural face ao SNMP.

8. Comparação direta com o SNMP. Se ainda tiver o agente SNMP da Aula 12 acessível, obtenha o nome do sistema pelas duas vias e compare o esforço e a legibilidade:

```
$ snmpget -v2c -c icnturma <IP> 1.3.6.1.2.1.1.5.0
```

Registe as duas formas de chegar à mesma informação.

4 Parte 2 – Gestão por API REST: o Proxmox (opcional)

Esta parte liga a matéria da aula à tendência atual discutida na teoria: a substituição de SOAP/XML por APIs REST com JSON.

1. **Consultar a API do Proxmox** a partir da VM, usando o token da Aula 07 (ou credenciais de leitura fornecidas pela docente):

```
$ curl -k -H "Authorization: PVEAPIToken=<user>@pve!<token>=<segredo>" \
https://<IP-do-servidor>:8006/api2/json/nodes
```

2. **Consultar o estado de um node** e as suas métricas:

```
$ curl -k -H "Authorization: PVEAPIToken=..." \
https://<IP-do-servidor>:8006/api2/json/nodes/<node>/status
```

3. **Comparar os três modelos de gestão** que usou ao longo do semestre para obter informação de um sistema:

```
SNMP -> snmpget sobre um OID numerico (Aula 12)
WBEM -> EnumerateInstances sobre uma classe (Parte 1)
REST -> GET sobre um URL, resposta em JSON (esta parte)
```

Registe, para cada um, como se identifica o objeto a consultar e em que formato vem a resposta.

5 Parte 3 – Roteiro de Revisão do Semestre

5.1 Introdução

Esta segunda parte não introduz matéria nova: é um roteiro de autoavaliação para consolidar, de forma aplicada, os conceitos centrais de cada bloco do programa, como preparação direta para o teste escrito.

5.2 Checklist por Bloco

- **C1 (Aulas 01-02):** Rever a distinção entre os Tiers de datacenter e o respetivo SLA associado. Ser capaz de explicar, por palavras próprias, porque é que redundância de energia/rede aumenta o Tier.
- **C2 (Aulas 03-04):** Comparar hipervisores tipo 1 vs tipo 2 e explicar porque é que um container partilha o kernel do host enquanto uma VM não. Rever o papel de namespaces e cgroups.
- **C3 (Aulas 05-07):** Repetir mentalmente o ciclo de vida de um Pod no Kubernetes até ao self-healing. Explicar a separação control/data plane da SDN. Distinguir gestão de configuração (Ansible) de provisionamento declarativo (Terraform).
- **C4 (Aulas 08-09):** Comparar scale-up vs scale-out e round-robin vs least connections. Explicar a diferença entre redundância ativa-ativa e ativa-passiva, e entre comunicação síncrona e assíncrona.
- **C5 (Aulas 10-11):** Distinguir block, file e object storage com um exemplo de cada. Explicar RAID 1 vs RAID 5 e a diferença entre NAS e SAN. Enunciar o teorema CAP ao nível introdutório.
- **C6 (Aulas 12-13):** Comparar a arquitetura manager/agente do SNMP com a arquitetura CIMOM/provider do WBEM. Identificar uma vantagem do WS-Management sobre o CIM-XML clássico.

5.3 Exercício de Síntese

Escolher *um* cenário de infraestrutura (ex. “uma aplicação web com picos de tráfego sazonais”) e, em poucas frases, descrever que tecnologias de cada bloco C1-C6 seriam relevantes para o desenhar e operar. Este exercício não é entregue, mas é fortemente recomendado como preparação para as questões de aplicação prática do teste escrito.

6 Entregável / Questões de Verificação

- a) Enviar os scripts usados e a lista de propriedades obtidas para a instância escolhida.
- b) Submeta a lista de associações encontradas (**ReferenceNames** / **AssociatorNames**). Escolha uma e explique, por palavras próprias, que relação do mundo real representa.
- c) Porque é que uma navegação por associações como esta não tem equivalente direto no SNMP? O que teria de fazer para obter a mesma informação a partir de uma MIB?
- d) A classe que inspecionou define *métodos*. Que diferença conceptual representa isto face às operações GET/SET do SNMP?
- e) Compare as três formas de obter o nome de um sistema que usou no semestre (SNMP, WBEM e, se fez a Parte 2, REST). Como se identifica o objeto em cada caso e em que formato vem a resposta?
- f) Que vantagem prática (não apenas teórica) sentiu ao consultar um CIMOM com **pywbem** em comparação com o **snmpwalk** da Aula 12? E que desvantagem?
- g) Se tivesse de escolher hoje uma tecnologia para gerir uma frota de servidores heterogêneos, qual escolheria e porquê? Considere SNMP, WBEM/WS-Management e APIs REST.